

Tutorial 11: Control Hazards in Pipelined Processors

ES 336 – Computer Organization and Architecture

Indian Institute of Technology Gandhinagar

Problem 1

Consider a non-pipelined processor operating at 2.5 GHz. It takes 5 clock cycles to complete an instruction. You are going to build a 5-stage pipeline from this processor. Overheads associated with pipelining force you to operate the pipelined processor at 2 GHz.

In a given program, assume that:

- 30% are memory instructions,
- 60% are ALU instructions,
- The remaining are branch instructions.

Further assume:

- 5% of the memory instructions cause 50-cycle stalls due to cache misses,
- 50% of the branch instructions cause 2-cycle stalls,
- There are no stalls associated with ALU instructions.

For this program, compute the speedup of the pipelined processor relative to the non-pipelined processor.

Solution to Problem 1

Non-pipelined.

- Clock period = $1/(2.5 \times 10^9) = 0.4$ ns.
- Cycles per instruction = 5 $\rightarrow$ time per instruction = $5 \times 0.4 = 2$ ns.

Pipelined.

- Clock period = $1/(2 \times 10^9) = 0.5$ ns.
- Base CPI = 1.

Stall cycles per instruction.

- **Memory stalls:**
 - Memory instructions = 30%. 5% cause 50-cycle stalls.
 - Fraction of instructions that cause memory stall = $0.3 \times 0.05 = 0.015$.
 - Extra cycles from memory = $0.015 \times 50 = 0.75$.
- **Branch stalls:**
 - Branch instructions = $100\% - 30\% - 60\% = 10\%$.
 - 50% cause 2-cycle stalls $\rightarrow$ fraction = $0.1 \times 0.5 = 0.05$.
 - Extra cycles from branches = $0.05 \times 2 = 0.1$.
- **Total extra cycles per instruction** = $0.75 + 0.1 = 0.85$.
- Effective CPI = $1 + 0.85 = 1.85$.
- Time per instruction = $1.85 \times 0.5 = 0.925$ ns.

Speedup.

$$\text{Speedup} = \frac{\text{time per instruction (non-pipe)}}{\text{time per instruction (pipe)}} = \frac{2}{0.925} \approx \mathbf{2.162}.$$

Problem 2

A pipelined processor with a base CPI of 1 uses a static branch prediction strategy. The processor executes a program containing a loop that iterates 100 times. In each iteration, a conditional branch instruction is executed at the end of the loop. The branch is taken for the first 99 iterations and not taken in the final iteration when the loop exits. The processor incurs a 3-cycle misprediction penalty whenever the branch prediction is incorrect.

Assume that:

- Branch instructions constitute 20% of the total instructions executed in the program.

- Apart from control hazards, there are no other stalls (no data or structural hazards).

Now consider two cases:

1. The processor always predicts the branch as **not taken**.
2. The processor always predicts the branch as **taken**.

Questions:

- (a) Calculate the total number of mispredictions in each case.
 - (b) Compute the effective CPI for both prediction strategies.
 - (c) Determine which prediction strategy gives better performance and justify your answer.
 - (d) If the loop is placed inside another loop (nested loop scenario), explain qualitatively how the misprediction behavior and CPI would change.
-

Solution to Problem 2

Step 1 – Total instructions.

- The loop runs 100 times, each iteration executes one branch $\rightarrow$ total branches = 100.
- Branches are 20% of all instructions $\rightarrow 0.2 \times N = 100 \rightarrow N = 500$ total instructions.

(a) Mispredictions.

- **Predict not taken:** correct only when branch is actually not taken (iteration 100). Mispredictions = 99.
- **Predict taken:** correct when branch is taken (iterations 1–99). Mispredictions = 1.

(b) Effective CPI.

- Base CPI = 1. Penalty per misprediction = 3 cycles.
- Predict not taken: total cycles = $500 + 99 \times 3 = 797 \rightarrow \text{CPI} = 797/500 = 1.594$.
- Predict taken: total cycles = $500 + 1 \times 3 = 503 \rightarrow \text{CPI} = 503/500 = 1.006$.

(c) Better strategy.

- Predict taken gives CPI 1.006 vs 1.594 for predict not taken. Since the branch is taken 99% of the time, predict taken is much better.

(d) Nested loop.

- Suppose the inner loop (100 iterations) is placed inside an outer loop that runs M times. Then:
 - The inner loop body (including its branch) executes $100M$ times.
 - Total branches = $100M$, and total instructions = $500M$ (branches still 20%).
 - Branch behavior per inner loop: taken 99 times, not taken once (at exit).
 - **Predict taken:** mispredictions occur only on the last iteration of each inner loop, i.e., M mispredictions. Total cycles = $500M + M \times 3 = 503M$. CPI = $503M/500M = 1.006$ (same as single loop case).
 - **Predict not taken:** mispredictions occur on the first 99 iterations of each inner loop, i.e., $99M$ mispredictions. Total cycles = $500M + 99M \times 3 = 797M$. CPI = $797M/500M = 1.594$ (also unchanged).
 - **Conclusion:** The CPI remains constant regardless of the outer loop count M . The “predict taken” strategy still outperforms “predict not taken” by the same margin, because the ratio of mispredictions to total instructions stays the same.
-

Problem 3

Consider the following MIPS assembly code fragment:

```
li    $t0, 0
li    $t1, 10
```

Loop:

```
    beq  $t0, $t1, Exit
    addi $t0, $t0, 1

    andi $t2, $t0, 1
    beq  $t2, $zero, Skip

    add  $s0, $s0, $t0
```

Skip:

j Loop

Exit:

Assume the following:

- A standard 5-stage MIPS pipeline is used.
- Branch instructions are resolved in the EX stage.
- Jump instructions are resolved in the ID stage.
- The branch prediction strategy is **always taken**.
- The misprediction penalty for branches is 2 cycles.
- The penalty for jump instructions is 1 cycle.
- There are no structural hazards.
- Data hazards are handled using full forwarding.

Answer the following questions:

(a) Determine the number of times each of the following branch instructions is executed:

- `beq $t0, $t1, Exit`
- `beq $t2, $zero, Skip`

(b) For each of the above branch instructions, determine:

- The number of correct predictions
- The number of mispredictions

(c) Compute the total number of clock cycles lost due to branch mispredictions.

(d) Compute the total number of clock cycles lost due to jump instructions.

Solution to Problem 3

(a) Number of executions of branch instructions

- `beq $t0, $t1, Exit`

The loop runs with `t0` from 0 to 9, and one final check occurs when `t0 = 10`.

Hence, total executions = $10 + 1 = 11$.

- `beq $t2, $zero, Skip`

This branch executes once per loop iteration (for `$t0 = 1` to `10`).

Hence, total executions = 10.

(b) Prediction analysis (Always Taken)

- `beq $t0, $t1, Exit`

Actual behavior: Taken once (when `$t0 = 10`), Not taken 10 times.

Prediction: Always taken.

– Correct predictions = 1

– Mispredictions = 10

- `beq $t2, $zero, Skip`

`$t2 = $t0 & 1`, so branch is taken for even values of `$t0`.

From `$t0 = 1` to `10`:

– Taken (even numbers: 2,4,6,8,10) = 5

– Not taken (odd numbers) = 5

Prediction: Always taken.

– Correct predictions = 5

– Mispredictions = 5

(c) Clock cycles lost due to branch mispredictions

Total mispredictions:

$$10 + 5 = 15$$

Penalty per misprediction = 2 cycles

$$\text{Total cycles lost} = 15 \times 2 = 30$$

(d) Clock cycles lost due to jump instructions

- j Loop

The jump instruction is executed once per iteration.

Total iterations = 10

Penalty per jump = 1 cycle

$$\text{Total cycles lost} = 10 \times 1 = 10$$

Problem 4

A pipelined processor employs a Branch History Table (BHT) with a single entry. As a result, all branch instructions map to the same predictor entry, causing complete aliasing. Two branch instructions, A and B, execute in strict alternation in the order A, B, A, B, ... Branch A is always taken, and Branch B is always not taken. Assume a base CPI of 1.

- The processor uses a **1-bit branch predictor**. Determine the steady-state prediction accuracy (after initial transient behavior).
 - The processor now uses a **2-bit saturating counter predictor** initialized to the **Strongly Not Taken** state. Determine the steady-state prediction accuracy and briefly explain the state transitions.
 - Compare the steady-state prediction accuracy of the 1-bit and 2-bit predictors for this scenario. Does the 2-bit predictor offer any improvement? If the performance depends on the initial state of the predictor, clearly justify your answer.
 - Using the 2-bit predictor from part (b), compute the effective CPI in steady state. Assume a misprediction penalty of 2 clock cycles and that every instruction executed is a branch instruction.
-

Solution to Problem 4

(a) 1-bit predictor.

- The predictor stores one bit: 0 = not taken, 1 = taken. After each branch, the bit is updated to the actual outcome.

- Trace:
 - Start with any bit.
 - A (taken): if bit=0 $\rightarrow$ mispredict, set to 1.
 - B (not taken): if bit=1 $\rightarrow$ mispredict, set to 0.
 - Repeat.
- Every branch mispredicts $\rightarrow$ **accuracy = 0%**.

(b) 2-bit saturating counter (initial SNT = 00).

- States: 00 (SNT), 01 (WNT), 10 (WT), 11 (ST). Prediction: taken if state ≥ 10 .
- Trace:

Branch	Current state	Prediction	Outcome $\rightarrow$ new state
A	00 (SNT)	not taken	taken $\rightarrow$ 01 (WNT)
B	01 (WNT)	not taken	not taken $\rightarrow$ 00 (SNT)
A	00	not taken	taken $\rightarrow$ 01
B	01	not taken	not taken $\rightarrow$ 00

- A always mispredicted, B always correct $\rightarrow$ **accuracy = 50%**.

(c) Comparison.

- 2-bit gives 50% accuracy, 1-bit gives 0% $\rightarrow$ 2-bit is better.
- Does initial state matter? If we start with ST (11):
 - A: predict taken $\rightarrow$ correct $\rightarrow$ stay at 11.
 - B: predict taken $\rightarrow$ mispredict $\rightarrow$ decrement to 10.
 - A: predict taken $\rightarrow$ correct $\rightarrow$ increment to 11.
 - B: predict taken $\rightarrow$ mispredict $\rightarrow$ decrement to 10.

Accuracy = 50% (A correct, B wrong). So steady-state accuracy is always 50% regardless of initial state.

(d) Effective CPI with 2-bit predictor.

- Every instruction is a branch. Misprediction rate = 0.5.
- Penalty = 2 cycles.
- Extra cycles per instruction = $0.5 \times 2 = 1$.
- CPI = base CPI + extra = $1 + 1 = 2$.

Problem 5

A processor X1 operating at 2 GHz has a standard 5-stage RISC instruction pipeline with a base CPI (cycles per instruction) of 1, assuming no pipeline hazards. For a given program P, 30% of the instructions are branch instructions. In X1, each branch instruction incurs a 2-cycle stall due to control hazards. A new version of the processor, X2, operates at the same clock frequency but includes a Branch Predictor Unit (BPU) that completely eliminates stalls for correctly predicted branches. There is neither any savings nor any additional stalls for other instructions. Assume that there are no structural hazards and no data hazards in both X1 and X2. The BPU has a prediction accuracy of 80%. Compute the speedup (rounded to two decimal places) obtained by X2 over X1 in executing program P.

Solution to Problem 5

X1 (no predictor).

- Base CPI = 1.
- Branch fraction = 0.3, penalty = 2 cycles.
- Extra CPI = $0.3 \times 2 = 0.6$.
- $\text{CPI}_{X1} = 1 + 0.6 = 1.6$.

X2 (with BPU).

- Prediction accuracy = 80% $\rightarrow$ misprediction rate = 0.2.
- Extra CPI = $0.3 \times 0.2 \times 2 = 0.12$.
- $\text{CPI}_{X2} = 1 + 0.12 = 1.12$.

Speedup.

- $\text{Speedup} = \frac{\text{CPI}_{X1}}{\text{CPI}_{X2}} = \frac{1.6}{1.12} \approx 1.42857$.
 - Rounded to two decimals: **1.43**.
-

Problem 6

Consider a pipelined processor with the following stages:

- IF (Instruction Fetch), ID (Decode and Operand Fetch), EX (Execute), WB (Write Back)
- IF, ID, and WB stages take 1 clock cycle each

The EX stage takes:

- ADD, SUB $\rightarrow$ 1 cycle
- MUL $\rightarrow$ 3 cycles
- DIV $\rightarrow$ 5 cycles

Assume operand forwarding is available unless stated otherwise.

(a) For the instruction sequence:

```
add $t2, $t1, $t0
mul $t4, $t3, $t2
sub $t6, $t5, $t4
```

Determine the total number of clock cycles required to execute the sequence.

(b) Assume a program consists of 20 instructions with the following distribution:

- 10 ADD instructions
- 3 SUB instructions
- 7 MUL instructions

Determine:

- Total number of clock cycles required
- CPI (Cycles Per Instruction)

(c) Consider the instruction sequence:

```
ADD, SUB, MUL, SUB, MUL, DIV, DIV
```

Assume:

- No data dependencies
- Execution times as given above

Determine the total number of clock cycles required, considering structural hazards.

(d) Consider the instruction sequence:

ADD, MUL, SUB, ADD, MUL, MUL, SUB, SUB, MUL

Assume:

- Each instruction (except the first) depends on the previous instruction
- No operand forwarding is used

Determine the total number of clock cycles required, considering data hazards and pipeline stalls.

Solution to Problem 6

(a) Sequence with forwarding.

- ADD → MUL dependency: forwarded, no stall.
- MUL (3-cycle EX) → SUB dependency: result ready after MUL's third EX cycle. SUB's EX can start the next cycle.
- Timeline:

Cycle:	1	2	3	4	5	6	7	8
	IF1	ID1	EX1	WB1				
		IF2	ID2	EX2	EX2	EX2	WB2	
			IF3	ID3		EX3	WB3	

- Total cycles = 8.

(b) 20 instructions, no dependencies, structural hazard on EX unit.

- Sum of EX latencies = $10 \times 1 + 3 \times 1 + 7 \times 3 = 10 + 3 + 21 = 34$.
- Total cycles = first instruction's IF+ID (2) + sum EX latencies + last WB (1) = $3 + 34 = 37$.
- CPI = $37/20 = 1.85$.

(c) 7 instructions, no dependencies, EX latencies: 1,1,3,1,3,5,5.

- Sum EX latencies = 19.
- Total cycles = $3 + 19 = 22$.

(d) 9 instructions, each depends on previous, no forwarding.

- Let S_i = cycle when instruction i starts EX.
- $S_1 = 3$.
- For $i > 1$: $S_i = S_{i-1} + L_{i-1} + 1$ (after previous's WB).
- Given $L = [1, 3, 1, 1, 3, 3, 1, 1, 3]$.

$$S_1 = 3$$

$$S_2 = 3 + 1 + 1 = 5$$

$$S_3 = 5 + 3 + 1 = 9$$

$$S_4 = 9 + 1 + 1 = 11$$

$$S_5 = 11 + 1 + 1 = 13$$

$$S_6 = 13 + 3 + 1 = 17$$

$$S_7 = 17 + 3 + 1 = 21$$

$$S_8 = 21 + 1 + 1 = 23$$

$$S_9 = 23 + 1 + 1 = 25$$

- WB of last = $S_9 + L_9 = 25 + 3 = 28$.
- Total cycles = 28, CPI = $28/9 \approx 3.11$.

Problem 7

Suppose that the branch frequencies (as percentages of all instructions) are as follows:

- Conditional branches: 15%
- Jumps and calls (unconditional branches): 1%
- Of the conditional branches, 60% are taken

- (a) We are examining a four-deep pipeline where the branch is resolved at the end of the second cycle for unconditional branches and at the end of the third cycle for conditional branches.

Assume that only the first pipeline stage can always be completed independent of whether the branch is taken, and ignore all other pipeline stalls.

Question: How much faster would the machine be without any branch hazards?

- (b) Now assume a high-performance processor with a 15-deep pipeline where the branch is resolved at the end of the fifth cycle for unconditional branches and at the end of the tenth cycle for conditional branches.

Assume that only the first pipeline stage can always be completed independent of whether the branch is taken, and ignore all other pipeline stalls.

Question: How much faster would the machine be without any branch hazards?

Solution to Problem 7

Let total instructions = N .

Branch counts.

- Conditional branches = $0.15N$. Of these, 60% taken $\rightarrow$ taken cond = $0.09N$, not-taken cond = $0.06N$.
- Unconditional branches (jumps/calls) = $0.01N$ (always taken).
- Total taken branches = $0.09N + 0.01N = 0.10N$.

(a) 4-deep pipeline.

- Resolution: unconditional at end of cycle 2 $\rightarrow$ penalty = 1 cycle (flush stage 2).
- Conditional taken at end of cycle 3 $\rightarrow$ penalty = 2 cycles (flush stages 2,3).
- Conditional not-taken: no penalty.
- Total penalty cycles = $0.01N \times 1 + 0.09N \times 2 = 0.01N + 0.18N = 0.19N$.
- Ideal CPI = 1 $\rightarrow$ actual CPI = $1 + 0.19 = 1.19$.
- Speedup without hazards = $1.19/1 = 1.19$ (19% faster).

(b) 15-deep pipeline.

- Resolution: unconditional at end of cycle 5 $\rightarrow$ penalty = 4 cycles.
- Conditional taken at end of cycle 10 $\rightarrow$ penalty = 9 cycles.
- Total penalty = $0.01N \times 4 + 0.09N \times 9 = 0.04N + 0.81N = 0.85N$.
- Actual CPI = $1 + 0.85 = 1.85$.
- Speedup without hazards = 1.85 (85% faster).

Problem 8

You have your dream (nightmare?) job as the head of the Architecture division of a cell-phone manufacturer that uses the classic 5-stage MIPS pipelined processor. An analysis of the programs that run on your cell-phones reveals many load-use hazards of the following special type: a `lw` instruction followed by a branch instruction that uses the value read from memory by the `lw`. For example:

```
lw $t0, 0($a0)
beq $t0, $t1, Label
```

The problem with such sequences is that the pipeline is stalled (for at least one cycle) even when the branch outcome is correctly predicted. Your colleague wants the company to switch to a 6-stage MIPS pipelined processor with the following pipeline stages: IF, ID, EX, MEM, BR, WB. The new stage, BR, is the Branch Resolution stage in which both the target address and the resolution of the branch are computed.

- (a) Your colleague claims that all load-use hazards of the special type above can be eliminated with forwarding on the 6-stage pipeline. Convince yourself that she is right by writing a pipeline diagram that shows where forwarding must be performed.

Instruction	1	2	3	4	5	6	7	8
lw	IF	ID	EX	MEM	BR	WB		
beq		IF	ID	EX	MEM	BR	WB	

- (b) Your colleague also claims that the 6-stage pipelined processor does not require any fancy branch-prediction, because the optimal strategy is to not predict at all. Is she right?
- (c) A drawback of predicting not-taken in the proposed pipeline is that branches aren't resolved until after the predicted instructions reach the MEM stage. Why might this be a problem? Write a hazard detection equation to stall the pipeline to avoid this problematic case.
- (d) On the 6-stage pipeline, if the branch target was computed in the ID stage instead of the BR stage, which simple prediction strategy (no prediction, always predict taken, always predict not taken) would you recommend? Why?

Solution to Problem 8

(a) Forwarding. The `lw` instruction produces its data at the end of the MEM stage. In the 6-stage pipeline, the `beq` instruction requires its operands in the BR stage. Since MEM comes before BR, the data can be forwarded directly from the MEM/BR pipeline register to the BR stage, eliminating the stall.

(b) No. Even in a 6-stage pipeline, "not predicting" is equivalent to a stall. Predicting "not taken" is generally better as it allows the pipeline to potentially continue without bubbles if the branch is indeed not taken.

(c) Problem. If instructions following the branch reach the MEM stage before the branch is resolved in BR, they might modify memory or processor state (if the architecture allows it) before we know if they should have been fetched.

Hazard Detection: if (ID/EX.WriteMem or EX/MEM.WriteMem) then Stall.

(d) Predict Not Taken. If the branch target is known in ID, but the resolution occurs in BR, predicting "taken" would require fetching from the target immediately. However, since the resolution is still several stages away, "always predict not taken" is the simplest strategy to maintain flow while waiting for resolution.